Nama: Fa'iza Alfagading Sofwan

CODEX

Miscellaneous:

1. Sanity Check

Petunjuk: Selamat bertanding!

CODEX{mana_flag_gratis_yang_kau_janjikan_itu_wok}

Cara: ada pada petunjuk tersebut

Flag: CODEX{mana_flag_gratis_yang_kau_janjikan_itu_wok}

2. Baca Rules
petunjuk: Baca rules terlebih dahulu! Bukan di sini

Cara: Melihat rules yang ada di discord karena ada kata "Bukan di sini", kemudian scroll sampai ke atas dan pada barisan This is the start of the # 📜 |rules channel. Tedapat penutup berwarna abu abu dan tekan kemudain muncullah flagnya.

Flag: CODEX{aku_mau_join_poros_plis_kalo_udah_oprec}

Crypto:

1. The Tiny Exponent Incident

Petunjuk: ada sesuatu yang kecil tapi sangat berpengaruh.. kecil... serta terdapat file dengan nama Chall.txt

Cara: Mengidentifikasi terlebih dahulu bagaimana cara menyelesaikan urutan kode tersebut, pertama ditemukan e = 3, kemudian n1, n2, dan n3 setelah di cek ternyata ukurannya 2048-4096 bit, kemudian ada c1, c2, dan c3 yang menunjukkan isi yang sama, sehingga setelah mengetahui 3 hal tersebut dan dicari kemudian cocok dengan Hastad's Broadcast Attack yang memiliki teorima:

1. e ciphertext dengan e yang sama
2. e modulus berbeda (n_1, n_2, ..., n_e)
3. Message m memenuhi $m < n_i$ untuk setiap i
4. Modulus n_i saling coprime ($\gcd(n_i, n_j) = 1$ untuk $i \neq j$)

Dan saya menyelesaikannya dengan Chinese Remainder Theorem converter dengan python pada google colab dengan kode:

```
pip install gmpy2


pip install pycryptodome


from Crypto.Util.number import long_to_bytes

import gmpy2


e = 3


n1 =
140902557672337313315683733328602432364718543504693736802884564103191621195836764795912652062905662924434409911193012471615192995905105128406030114794806278114681629291704293831439595879830860491190496033594795302576600936185918218307472095921006387374407802465867544444039001362968327751246771557637170614851031238873510082921419558870799522935969730667010484498872281653941837026119779236802613695561797004550634394878326336773794465887817373319854663973633698618808664444060589359308483766235019128022528164916592350514414097080137347613607253562420309699047820591143458978387757476854736143127436628791583532222 09

c1 =
613286920856318917028620967785899526816206523772896215313601254402302837870777826665092464071606732318553382568546544906515085745446157860088305524343347135371553116626525301209652070149465851467638511928231991073596603300426117008164928253967354278923094805148930207878990172645974696217845755362553360620786578110445168009344220945151590565470551099024285005123745950478988429488962326821395082144831072732927654983262077714199150257852146697727315738223513168756472127782672376577532626859946746862337937550721731277923783955723920385538555196970646170923146 1

n2 =
235960262659619901537892965154739326242465599767068446189344764918795660996894563602186556365364644573192943163536307268829469571586223002214518587120137562296129185841131591888107745358139169165918574504507633070731690743
```

7262615331026160129221319925809252141113407756413816926898742967449455064672037500618099801990653478849133693820622461999160763271305381819953398424399233409538904571954601839801270712776297279632282825101198359981199004541160884699455723392264449234756875208173758157156644427549057845264977077115324675833789063094591620025560845958970629369654619627029907190466452971756318850211848938329501 9

c2 =
6132869208563189170286209677858995268162065237728962153136012544023028378707778266650924640716067323185533825685465449065150857454461578600883055243433471353715531166265253012096520701494658514676385119282319910735966033004261170081649282539673542789230948051489302078789901726459746962178457553625533606207865781104451680093442209451515905654705510990242850051237459504789884294889623268213950821448310727329276549832620777141991502578521466977273157382235131687564721277826723765775326268599467468623379375507217312779237839557239203855385551969706461709231461

n3 =
1551231044668762752245384365626125541248921245991606046975149759291205291654750203050550086279559601820840615638288180720539010429335927702824086625198850161589055981728776307077957069329332904498612280595374376398240907738263197974435838605023318810429148907924521976132115923929475763225683910342315849634077472988994746257263961923404188203062769109105057237100826623441455829456136096241939006257356195548473822350594623890079144875876314466126590469728744264652763309707558835682688048181798274640588994515284831104334790109423999965589928633314353528154450097578722457993065477819432196578165335964900633412354 9

c3 =
6132869208563189170286209677858995268162065237728962153136012544023028378707778266650924640716067323185533825685465449065150857454461578600883055243433471353715531166265253012096520701494658514676385119282319910735966033004261170081649282539673542789230948051489302078789901726459746962178457553625533606207865781104451680093442209451515905654705510990242850051237459504789884294889623268213950821448310727329276549832620777141991502578521466977273157382235131687564721277826723765775326268599467468623379375507217312779237839557239203855385551969706461709231461

def hastad_attack(c_list, n_list, e):

```
    from functools import reduce

    def chinese_remainder_theorem(c, n):
        total = 0
        prod = reduce(lambda a, b: a * b, n)

        for c_i, n_i in zip(c, n):
            p = prod // n_i
            total += c_i * gmpy2.invert(p, n_i) * p
        return total % prod

    hasil = chinese_remainder_theorem(c_list, n_list)

    m, exact = gmpy2.iroot(hasil, e)

    if exact:
        return long_to_bytes(int(m))
    else:
        return None

c_list = [c1, c2, c3]
n_list = [n1, n2, n3]

flag = hastad_attack(c_list, n_list, e)
print("Flag:", flag)
```

Flag:
CODEX{R54_e3_74NpA_pAdd1N9_YaN9_b354r???!!!_w4H_17U_k4caU_s1h_Ju7Ur_KAc4u_SiH}

2. 47 no Shinjutsu

Penjelasan: Seorang jurnalis misterius dari Tokyo mengirimkan berita penting, namun tulisannya tidak bisa dibaca. Bisakah kamu mengungkap kebenaran dari 47 dan membaca berita yang sebenarnya? Dan file dengan nama Chall.txt

Cara: Indentifikasi terlebih dahulu bahwa itu adalah kode hex yang berisi pesan tersembunyi dan ada kemungkinan encoding berlapis. Kemudian coba di decode, saya menggunakan cyberchef yang pertama menggunakan from hex kemudian ternyata masih menjadi kode aneh, setelah lihat soalnya ternyata ada angka 47, setelah dicari di pencarian pada operations ternyata ada ROT47 kemudian dicoba dan kode aneh tersebut dapat dibaca, dan pada teks tersebut terdapat flagnya.

Flag: CODEX{W0w_j3l1_jUg4_Lu_b4nG_nY4r1_fL49_nY4_n1Ch_9f73b43e683a73}

3. All roads lead to Rome

Penjelasan: You are running out of time dan diberikan file chall.enc serta file chall.py

Cara: Diberikan file chall.py untuk kerangka sebagai dasar membuat brute force logicnya, disini saya menggunakan IDEnya google colab, yaitu yang awalnya

```
#!/usr/bin/env python3

import hashlib, struct, sys

MAGIC = b"CTFPNG1"

def compute_keystream(secret: bytes, filesize: int, nbytes: int) -> bytes:

    out = bytearray()

    counter = 0

    while len(out) < nbytes:

        h = hashlib.sha256()

        h.update(secret)

        h.update(filesize.to_bytes(8, "big"))

        h.update(counter.to_bytes(8, "big"))

        out.extend(h.digest())

        counter += 1
```

```
    return bytes(out[:nbytes])
def parse_flag_enc(path: str):
    with open(path, "rb") as f:
        magic = f.read(len(MAGIC))
        if magic != MAGIC:
            raise ValueError("Bad file format")
        filesize = struct.unpack(">Q", f.read(8))[0]
        ciphertext = f.read()
        if len(ciphertext) != filesize:
            raise ValueError("Ciphertext length differs from recorded filesize")
        return filesize, ciphertext
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: chall.py flag.enc")
        sys.exit(2)
    filesize, ciphertext = parse_flag_enc(sys.argv[1])
    print("Keystream block = SHA256(secret || filesize || counter)")
    print("secret is NOT provided. Author used a 24-bit secret.")
    print("filesize:", filesize)
    print()
    print("Use compute_keystream(secret_bytes, filesize, nbytes) to generate keystream")
```

menjadi:

```
#!/usr/bin/env python3
import hashlib, struct, sys, os
from multiprocessing import Pool, cpu_count
import math
```

```
MAGIC = b"CTFPNG1"

def compute_keystream(secret: bytes, filesize: int, nbytes: int) -> bytes:
    out = bytearray()
    counter = 0
    while len(out) < nbytes:
        h = hashlib.sha256()
        h.update(secret)
        h.update(filesize.to_bytes(8, "big"))
        h.update(counter.to_bytes(8, "big"))
        out.extend(h.digest())
        counter += 1
    return bytes(out[:nbytes])

def parse_flag_enc(path: str):
    with open(path, "rb") as f:
        magic = f.read(len(MAGIC))
        if magic != MAGIC:
            raise ValueError("Bad file format")
        filesize = struct.unpack(">Q", f.read(8))[0]
        ciphertext = f.read()
        if len(ciphertext) != filesize:
            raise ValueError("Ciphertext length differs from recorded filesize")
        return filesize, ciphertext

# FUNGSI GLOBAL untuk multiprocessing
def check_secret_range(args):
    start, end, ciphertext, filesize = args
```

```
    CHECK_BYTES = 8

    for secret_int in range(start, end):
        secret = secret_int.to_bytes(3, 'big')
        keystream = compute_keystream(secret, filesize, CHECK_BYTES)

        # Quick PNG check (first 4 bytes)
        if (ciphertext[0] ^ keystream[0] == 0x89 and
            ciphertext[1] ^ keystream[1] == ord('P') and
            ciphertext[2] ^ keystream[2] == ord('N') and
            ciphertext[3] ^ keystream[3] == ord('G')):
            return secret_int

        # Quick flag check (first 6 bytes)
        if (ciphertext[0] ^ keystream[0] == ord('C') and
            ciphertext[1] ^ keystream[1] == ord('O') and
            ciphertext[2] ^ keystream[2] == ord('D') and
            ciphertext[3] ^ keystream[3] == ord('E') and
            ciphertext[4] ^ keystream[4] == ord('X') and
            ciphertext[5] ^ keystream[5] == ord('{')):
            return secret_int

    return None

def super_optimized_brute_force(ciphertext, filesize):
    print("🚀 Starting SUPER OPTIMIZED brute force (with multiprocessing)...")

    num_processes = cpu_count()
```

```
total_secrets = 0xFFFFFF + 1
chunk_size = math.ceil(total_secrets / num_processes)

print(f"🖥️ Using {num_processes} CPU cores...")
print(f"📊 Total secrets to check: {total_secrets:,}")
print(f"📦 Chunk size per process: {chunk_size:,}")

ranges = []
for i in range(num_processes):
    start = i * chunk_size
    end = min((i + 1) * chunk_size, total_secrets)
    ranges.append((start, end, ciphertext, filesize))

try:
    with Pool(num_processes) as pool:
        print("🎯 Starting parallel processing...")
        results = pool.map(check_secret_range, ranges)

        for i, result in enumerate(results):
            if result is not None:
                secret = result.to_bytes(3, 'big')
                print(f"✅ Found potential match in process {i+1}")

                # Verify dengan full decryption
                full_keystream = compute_keystream(secret, filesize, len(ciphertext))
                plaintext = bytes(c ^ k for c, k in zip(ciphertext, full_keystream))
```

```
            print(f"🎯 Secret found: {secret.hex()}")

            if plaintext.startswith(b'\x89PNG'):
                with open('decrypted.png', 'wb') as f:
                    f.write(plaintext)
                print("💾 Saved as decrypted.png")

                # Try to display if in Colab
                try:
                    from IPython.display import Image, display
                    display(Image(filename='decrypted.png'))
                except:
                    print("🖼️ PNG file saved successfully!")

            # Check for flag text
            flag_text = plaintext.decode('utf-8', errors='ignore')
            if 'CODEX{' in flag_text:
                start_idx = flag_text.index('CODEX{')
                end_idx = flag_text.index('}', start_idx) + 1
                print(f"🎉 FLAG FOUND: {flag_text[start_idx:end_idx]}")

            return secret.hex()

    print("❌ Secret not found in 24-bit range")
    return None

except Exception as e:
```

```
        print(f"❌ Multiprocessing error: {e}")
        print("🔄 Falling back to single thread...")
        return single_thread_brute_force(ciphertext, filesize)

def single_thread_brute_force(ciphertext, filesize):
    print("🔍 Starting single thread brute force...")

    CHECK_BYTES = 8

    for secret_int in range(0, 0xFFFFFF + 1):
        secret = secret_int.to_bytes(3, 'big')
        keystream = compute_keystream(secret, filesize, CHECK_BYTES)

        # PNG signature check
        if (ciphertext[0] ^ keystream[0] == 0x89 and
            ciphertext[1] ^ keystream[1] == ord('P') and
            ciphertext[2] ^ keystream[2] == ord('N') and
            ciphertext[3] ^ keystream[3] == ord('G')):

            # Found! Full decryption
            full_keystream = compute_keystream(secret, filesize, len(ciphertext))
            plaintext = bytes(c ^ k for c, k in zip(ciphertext, full_keystream))

            print(f"✅ Found with secret: {secret.hex()}")

            with open('decrypted.png', 'wb') as f:
                f.write(plaintext)
            print("💾 Saved as decrypted.png")
```

```
            # Display if in Colab
            try:
                from IPython.display import Image, display
                display(Image(filename='decrypted.png'))
            except:
                pass

            return secret.hex()

        # Progress indicator
        if secret_int % 1000000 == 0 and secret_int > 0:
            progress = (secret_int / 0xFFFFFF) * 100
            print(f"📊 Progress: {secret_int:,}/16,777,216 ({progress:.2f}%)")

    print("❌ Secret not found")
    return None

# Main execution
print("🔐 CTF PNG Decryption Challenge - FIXED VERSION")
print("=" * 50)

# Cek jika file chall.enc sudah ada
if not os.path.exists('chall.enc'):
    print("📁 Please upload chall.enc file")

    # Untuk Google Colab
    try:
```

```
    from google.colab import files
    uploaded = files.upload()

    if not uploaded:
        print("❌ No file uploaded.")
    else:
        for filename in uploaded.keys():
            if filename.endswith('.enc'):
                try:
                    filesize, ciphertext = parse_flag_enc(filename)
                    print("✅ File parsed successfully!")
                    print(f"📏 File size: {filesize} bytes")

                    # Gunakan multiprocessing version
                    found_secret = super_optimized_brute_force(ciphertext, filesize)

                    if found_secret:
                        print(f"🎉 Success! Secret key: {found_secret}")
                    else:
                        print("💥 Failed to find secret")

                except Exception as e:
                    print(f"❌ Error processing {filename}: {e}")
    except ImportError:
    print("❌ Please put chall.enc in the same directory")
    print("Usage: python3 decrypt.py chall.enc")
else:
```

```
    # File already exists
    try:
        filesize, ciphertext = parse_flag_enc('chall.enc')
        print("✅ File parsed successfully!")
        print(f"📏 File size: {filesize} bytes")

        # Gunakan multiprocessing version
        found_secret = super_optimized_brute_force(ciphertext, filesize)

        if found_secret:
            print(f"🎉 Success! Secret key: {found_secret}")
        else:
            print("💥 Failed to find secret")

    except Exception as e:
        print(f"❌ Error processing chall.enc: {e}")
```

Setelah menunggu beberapa saat akan keluar flagnya dalam bentuk gambar.

Flag: CODEX{All_r04d5_le4d_t0_rom3_but_my_ch4ll_l3ads_to_c0d3x_champ}

Osint:

1. Night Out

Penjelasan: I found this old photo in my gallery. I think this was taken during a night out around June 9, 2020 but I totally forgot where exactly this was. Can you help me find the exact spot where I took this photo?

what3words link: https://what3words.com/

Note:

- ● Change your what3words language to English in the settings.
- ● The what3words landmark is the exact position where the photo was taken.

Flag Format: CODEX{word1_word2_word3}

Cara: upload foto tersebut ke google lens, kemudian tahu kalo foto tersebut berada di bandung, dan dilihat dari fotonya terdapat tulisan braga pada bangunannya sehingga berada di Jl. Naripan No.9-21, Braga, Kec. Sumur Bandung, Kota Bandung, Jawa Barat 40111 setelah itu kita cocokkan posisi gambar tersebut dengan posisi di maps pada link untuk mencari what3wordsnya, yaitu pada kata: ///dictation.pavement.panning sehingga ditemukan flagnya.

Flag: CODEX{dictation_pavement_panning}

2. Intro to Underworld

Penjelasan: Halo semua, untuk menyelam ke dunia yang tidak terjamah atau lebih tepatnya untuk menyelesaikan soal kali ini, tidak bisa menggunakan cara biasa yaaa!!. Salah satu cara yang bisa dipakai adalah menggunakan Tor. Berikut cara instalasinya:

Cara Install TOR di Windows https://www.youtube.com/watch?v=5EZJq1BeZ6w

Cara Install TOR di Linux https://www.youtube.com/watch?v=Bw6lI01hAKg

Jika sudah, pada peramban tor atau tor browser, masuk ke MarsMarket. Jika kalian tidak tahu MarsMarket, bisa langsung buka link berikut pada tor browser : marshjhofnon25wotp27mwmln62adlkbb55wmhrgt6b7tabjimqnj7yd.onion

Jangan lupa buat akun terlebih dahulu. Lalu, cari namaku, "Pendongeng" di MarsMarket. (Cari aja, gampang kok)

Cara: Install browser TOR terlebih dahulu, setelah terinstall buka dan cari marshjhofnon25wotp27mwmln62adlkbb55wmhrgt6b7tabjimqnj7yd.onion

Setelah itu daftar dan ketika sudah selesai daftar di pencarian tambahkan /users/Pendongeng jadi untuk link lengkapnya adalah:

http://marshjhofnon25wotp27mwmln62adlkbb55wmhrgt6b7tabjimqnj7yd.onion/users/Pendongeng dan muncullah flagnya

Flag: CODEX{w3lcome_t0_und3rw0rld?!_dont_tru5t_4nyon3}

3. Dongeng yang Hilang

Penjelasan: Dari 1000 suara yang telah kuraih... Maksudnya 1000 dunia yang telah kujelajahi, tidak semata-mata hanya karena tujuan pribadi. Semua kulakukan untuk meninggalkan sebuah dongeng di tiap kehidupan yang kusinggahi.

Pada kehidupan atau dunia kali ini, aku hidup sebagai salah satu Staff Penerima Tamu di Universitas Brawijaya. Tepatnya aku bekerja di Pusat Layanan Terpadu Universitas Brawijaya (PLT UB) atau lebih detailnya unit yang lebih kecil, yaitu ULT UB Fakultas Ilmu Komputer. Aku diberikan sebuah buku berisi tamu yang datang dan keluar. Tugasku tidak lain adalah mencatat pada buku tamu. Kalian tahu? Jenis bukunya aneh, soalnya tidak pakai kertas, cukup lihat layar saja. Lebih aneh lagi, setiap hari berganti, semua catatan tamu akan terhapus dengan sendirinya. Praktis sekali, bukan? ✨

Setelah cukup menikmati dunia ini, tibalah waktunya aku pergi. Sebagai tanda terima kasih pada dunia yang kusinggahi kali ini, kutinggalkan sebuah dongeng di buku tamu itu. Tenang saja, walaupun dongeng yang kutulis tersebut hilang bersama hari berganti, dongeng tersebut akan kembali muncul bersama dengan namaku pada pukul 00.01 AM.

"Dongeng yang kutinggalkan adalah sebuah **rahasia yang hilang**. Untuk membacanya, kalian perlu mengetahui bagaimana cara kerja dunia kalian"

Cara: pada penjelasan diatas terdapat ” Pada kehidupan atau dunia kali ini, aku hidup sebagai salah satu Staff Penerima Tamu di Universitas Brawijaya. Tepatnya aku bekerja di Pusat Layanan Terpadu Universitas Brawijaya (PLT UB) atau lebih detailnya unit yang lebih kecil, yaitu ULT UB Fakultas Ilmu Komputer. Aku diberikan sebuah buku berisi tamu yang datang dan keluar. Tugasku tidak lain adalah mencatat pada buku tamu. Kalian tahu? Jenis bukunya aneh, soalnya tidak pakai kertas, cukup lihat layar saja. Lebih aneh lagi, setiap hari berganti, semua catatan tamu akan terhapus dengan sendirinya. Praktis sekali, bukan?” dari teks tersebut bisa diketahui kalau akan menggunakan web https://plt.ub.ac.id/ kemudian pilih buku tamu sehingga linknya https://bukutamu.ub.ac.id/landing kemudian ada petunjuk selanjutnya yaitu ” ULT UB Fakultas Ilmu Komputer” setelah berbagai percobaan, linknya menjadi https://bukutamu.ub.ac.id/landing/ult-filkom dan muncul pesan yang memiliki detail jamnya 00:01 dan atas nama Pendongeng, setelah itu inspect element kemudian ke network dan tekan nama kunjungan kemudian tekan kebawah semua dan di bagian keperluan muncul ”Kuberikan salah satu dongengku : pastebin.com/D0caiNA1 Jika kau tetap tidak bisa membaca dongengku, maka kau terlambat. Namun, jikalau kau terlambat, kau hanya perlu kembali ke masa lalu untuk membacanya”. Setelah dicek di pastebin.com/D0caiNA1 ternyata **Not Found (#404)**

This page is no longer available. It has either expired, been removed by its creator, or removed by one of the Pastebin staff.

Jadi saya menggunakan https://web.archive.org/ untuk melihat ke masa lalu tempel link pastebin.com/ D0caiNA1 kemudian enter dan pada bagian tahun di 2025, bulannya November, tanggal 9 tekan yang 10:10:48 dan muncullah flagnya.

Flag: CODEX{k0mb1n4s1_k3cil_os1nt_w3b_for3ns1c!}

Forensics:

1. sunshine

Penjelasan: *You are my sunshine*
My only sunshine
You make me happy
When skies are gray
You'll never know, dear
How much I love you
Please don't take
My sunshine away

Dan file sunshine.png

Cara: buka wsl dan pindah file sunshine tersebut ke penyimpannya ubuntu yang digunakan kemudian masuk ke folder yang ada file sunshine tersebut pada wslnya dengan cara cd nama folder, kemudian install foremost dengan cara sudo apt install -y foremost, setela selesai ketik foremost -i sunshine.png -o extracted_files/. Setelah selesai buka file yang Namanya extracted_files, didalamnya terdapat folder pdf, png, dan audit.txt, buka folder pdfnya dan buka pdf tersebut, kemudian keluarlah flagnya.

Flag: CODEX{9634e9ev9j99dt9c0qr5https://tinyurl.com/akuForensics}

Reverse:

1. The Packed Grimoire

Penjelasan: In an age when knowledge was sealed behind arcane codes, a mysterious program known as The Packed Grimoire emerged. It’s said that this enchanted binary holds a spell that only reveals its truth to those who know the right incantation. Dare to unseal it, and prove your worth to the archive of forgotten flags.

Cara: gunakan wsl kemudian pertama buka permissionnya dengan cara ”chmod +x chall” dan enter. Kemudian ketikkan ”./chall &” untuk menjalankan secara memori dinamis analisis, kemudian muncul ”Enter flag:” masukkan ” PID=$!” dan enter. Setelah itu ketikkan ”sudo cat /proc/$PID/maps” dan enter untuk Dump memory mapping process selanjutnya ketikkan “sudo gcore -o core_dump $PID” untuk membuat core dump dari process yang sedang running. Kemudian untuk mendapatkan flagnya ketikkan ”strings core_dump.$PID | grep -E "CTF{.*}|flag{.*}|{.*}"” dan muncul flagnya

Flag: CODEX{S1mPl3_fl4G_ch3ckEr_4nD_p4ck3Rs5S5s!!!0_o!!!}